

SMAP

Simple Music Adaptive Protocol

จัดทำโดย

พันธุ์รัช พราวพันธุ์

รหัสนิสิต: 6710405109

Project: Socket Programming

วิชา: 01418351

สารบัญ

สารบัญ.....	2
1. วัตถุประสงค์ของ Protocol.....	3
2. คุณลักษณะของ Protocol (Characteristics).....	3
3. Transport Layer Service Model ที่ใช้และเหตุผล.....	3
4. การออกแบบ Application-Layer Protocol (SMAP).....	4
5. ค่าคงที่ของโปรโตคอล (Protocol Constants).....	4
6. คลาสหลักในการทำงานของโปรโตคอล (Core Classes).....	5
7. เมธอดและฟังก์ชันของโปรโตคอล (Protocol Methods & Functions).....	7
7.1 send_all(sock, data: bytes).....	7
7.2 receive_exact(buffered_sock, n: int) -> bytes.....	8
7.3 Request Helpers (ฝั่งสร้าง/แปลงคำขอ).....	8
7.4 Response Helpers (ฝั่งสร้าง/แปลงคำตอบ).....	9
7.5 to_json_bytes(obj) -> bytes.....	10
7.6 calculate_throughput_kbps(num_bytes, elapsed_seconds) -> float.....	10
7.7 select_bitrate(preferred_bitrate, available_throughput_kbps) -> (bitrate, reason).....	10
8. จุดเด่นและข้อเปรียบเทียบของ Protocol.....	11
8.1 Lightweight & Purpose-Built (ใช้ Overhead ต่ำกว่า HTTP).....	11
8.2 Stateful Connection ลดการสื่อสาร (Low Latency).....	11
8.3 Single Persistent Connection ลดเวลา TCP Handshake.....	11
8.4 Built-in Adaptive Bitrate (ABR) และวิธีการวัดผลที่เป็นรูปธรรม.....	11
9. ข้อจำกัดของ Protocol และระบบ (Limitations).....	12
10. การตรวจสอบการทำงานด้วย Wireshark.....	13

1. วัตถุประสงค์ของ Protocol

โปรโตคอล SMAP (Simple Music Adaptive Protocol) ถูกออกแบบมาเพื่อเป็น Application-Layer Protocol สำหรับการสตรีมเพลง (MP3) แบบ Adaptive-bitrate โดยเฉพาะผ่านการเชื่อมต่อ TCP เพียงเส้นเดียว (Persistent Connection) โดยมุ่งเน้นความเรียบง่าย (Lightweight) ลด Overhead ที่ไม่จำเป็นออก เพื่อให้การทำงานในฝั่ง Network มีประสิทธิภาพสูงสุด และสามารถปรับเปลี่ยนระดับคุณภาพเสียง (Bitrate) ตามสภาพเครือข่ายได้อย่างรวดเร็วและต่อเนื่อง

2. คุณลักษณะของ Protocol (Characteristics)

- Application-Layer Protocol: ทำงานบนชั้น Application ออกแบบเป็น Text-based Request/Response ทำให้อ่านและเข้าใจโครงสร้างได้ง่าย
- Persistent Connection: เปิดการเชื่อมต่อ TCP เพียงครั้งเดียว (Single Connection) และใช้งานต่อเนื่องสำหรับทุกๆ คำสั่งจนกว่าจะสั่ง QUIT
- Stateful Protocol: ฝั่ง Server จะมีการจดจำสถานะ (Session state) ของแต่ละ Client อย่างแยกขาดจากกัน เช่น ค่า Preferred Bitrate และ Network Simulation Profile ทำให้ไม่ต้องแนบสถานะเดิมส่งไปใหม่ในทุก Request
- Adaptive Bitrate (ABR) Support: รองรับการส่งไฟล์สื่อแบบแบ่งเป็นชิ้นย่อยๆ (Segments) ขนาดคงที่ที่ 8192 bytes (สามารถเปลี่ยนได้ใน protocol) เพื่อให้ Client สามารถประเมินแบนด์วิดท์และร้องขอคุณภาพเสียงที่เหมาะสมใน Segment ถัดไปได้

3. Transport Layer Service Model ที่ใช้และเหตุผล

โปรโตคอล SMAP เลือกใช้ TCP (Transmission Control Protocol)

เหตุผล: การสตรีมไฟล์เพลงเป็น Segment ย่อยๆ และนำมาประกอบกันนั้น จำเป็นต้องอาศัยความถูกต้องของข้อมูล (Reliability) และลำดับข้อมูลที่ถูกต้อง (In-order delivery) หากใช้ UDP ข้อมูลที่สูญหายเพียงเล็กน้อยหรือสลับลำดับอาจทำให้ไฟล์ MP3 เสียหายและเล่นไม่ได้ นอกจากนี้ TCP ยังมีกลไก Flow Control และ Congestion Control ที่ช่วยจัดการอัตราการส่งข้อมูลให้สอดคล้องกับแบนด์วิดท์ของระบบเครือข่าย

4. การออกแบบ Application-Layer Protocol (SMAP)

โครงสร้างถูกออกแบบโดยให้ความสำคัญกับตัว Protocol เป็นหลัก ส่วนตัวแอปพลิเคชัน (MiniMP3Stream) เป็นส่วนประกอบรองที่นำไปใช้ทดสอบการทำงานจริง:

- รูปแบบคำสั่ง (Request Format): ส่งเป็นข้อความบรรทัดเดียว (Text Line) โครงสร้างคือ COMMAND [param1] [param2] ...
คำสั่งหลักเช่น LIST, PLAY, GET_SEGMENT, BITRATE, NETWORK, STOP, QUIT
- รูปแบบการตอบกลับ (Response Format): มีรูปแบบคล้าย HTTP ประกอบด้วยบรรทัดสถานะ (STATUS_CODE STATUS_PHRASE) ตามด้วย Header fields, บรรทัดว่าง (\n\n) และปิดท้ายด้วย Body (เช่น Raw MP3 bytes ตามความยาวของ Content-Length)
- ส่วนของแอปพลิเคชัน (Application Design): ใช้สถาปัตยกรรมแบบ Thread-per-client เพื่อรองรับหลาย Client พร้อมกัน และมีฟังก์ชัน Network Simulation ภายในตัวเพื่อควบคุมและสังเกตผลกระทบต่อ ABR ได้อย่างชัดเจน

5. ค่าคงที่ของโปรโตคอล (Protocol Constants)

ไฟล์ protocol.py กำหนดค่าคงที่ (constants) ไว้ที่ส่วนบนของไฟล์ เพื่อให้ทั้ง server.py และ client.py อ้างอิงค่าเดียวกันเสมอ ป้องกันปัญหาค่าไม่ตรงกันระหว่างสองฝั่ง (เช่น ขนาด Segment ที่ Server ตัดกับที่ Client คาดหวัง) ตารางด้านล่างสรุปค่าคงที่ที่สำคัญต่อการทำงานของ ABR และของโปรโตคอลโดยรวม:

ค่าคงที่ (Constant)	ค่า (Value)	ความหมายและจุดประสงค์
SEGMENT_SIZE_BYTES	8192	ขนาด (bytes) ของ Segment แต่ละชิ้นที่ Server ตัดจากไฟล์ MP3 ต้นฉบับ (Fixed-size byte-range) เป็นหน่วยที่ Client ใช้ร้องขอที่ละชิ้นผ่านคำสั่ง GET_SEGMENT และเป็นหน่วยที่ใช้วัด Throughput ในแต่ละรอบ
THROUGHPUT_HISTORY_SIZE	3	จำนวน Segment ล่าสุดที่ Client เก็บค่าความเร็วดาวน์โหลด (Throughput) ไว้เพื่อคำนวณค่าเฉลี่ย ช่วยลดผลกระทบจากความผันผวนชั่วขณะของเครือข่าย (Noise) ก่อนตัดสินใจเลือก Bitrate ของ Segment ถัดไป

ค่าคงที่ (Constant)	ค่า (Value)	ความหมายและจุดประสงค์
SAFETY_FACTOR	0.70	ตัวคูณลดทอน (70%) ที่นำไปคูณกับค่า Throughput เฉลี่ยที่วัดได้ ก่อนนำไปเทียบเลือก Bitrate เพื่อเผื่อ Margin ไว้สำหรับความผันผวนของเครือข่ายในอนาคต ป้องกันอาการเสียงกระตุก (Buffer Underrun)
STATUS_PHRASES	dict {code: phrase}	ตารางแมประหว่างรหัสสถานะ (Status Code) กับข้อความสถานะ (Status Phrase) เช่น 200 → OK, 404 → NOT FOUND ใช้ตอนสร้าง Response Header ฝั่ง Server (ดูรายละเอียดในตารางถัดไป)

รายละเอียด STATUS_PHRASES (ตารางรหัสสถานะที่ SMAP รองรับ):

Status Code	Status Phrase	ความหมาย / กรณีใช้งาน
200	OK	คำขอสำเร็จ ใช้กับ LIST, PLAY, GET_SEGMENT, BITRATE, NETWORK, STOP ที่ทำงานถูกต้อง
400	BAD REQUEST	รูปแบบคำสั่ง (syntax) ไม่ถูกต้อง หรือ พารามิเตอร์ไม่ครบ/ไม่ถูกประเภท
404	NOT FOUND	ไม่พบเพลงหรือ Segment ที่ร้องขอในแคตตาล็อกของ Server
406	NOT ACCEPTABLE	ค่า Bitrate ที่ร้องขอไม่อยู่ใน [64, 128, 192, 320] หรือเพลงนั้นไม่มีไฟล์ที่ Bitrate นั้น
500	INTERNAL SERVER ERROR	ข้อผิดพลาดฝั่ง Server เช่น ไฟล์เพลงที่แคตตาล็อกอ้างอิงถูกลบ/ย้ายไปแล้ว

6. คลาสหลักในการทำงานของโปรโตคอล (Core Classes)

protocol.py มีทั้งหมด 3 คลาส โดยสองคลาสแรกเป็น Exception ที่ใช้สื่อสารข้อผิดพลาดระหว่างชั้นโปรโตคอลกับโค้ดที่เรียกใช้งาน (server.py / client.py) ส่วนคลาสที่สามคือแกนหลักที่แก้ปัญหาการอ่านข้อมูลจาก TCP stream:

class ProtocolError(Exception) — *Exception class*

ใช้แจ้งข้อผิดพลาดเมื่อข้อมูลที่ได้รับมาไม่เป็นไปตามรูปแบบ (format) ของโปรโตคอล SMAP เช่น ส่ง Request บรรทัดว่างมา หรือ Response Header ไม่มีบรรทัดสถานะ

Use case: server.py เรียก parse_request() แล้วพบว่า Client ส่งบรรทัดว่างมา ProtocolError จะถูก raise แล้วถูก server ดักจับเพื่อตอบกลับ 400 BAD REQUEST แทนที่จะปล่อยให้ Thread ของ Client นั้น Crash

```
try:
    command, params = parse_request(line)
except ProtocolError as e:
    send_response_header(sock, 400, {"Message": str(e)})
```

class ConnectionClosedError(Exception) — *Exception class*

ใช้แจ้งว่าอีกฝั่ง (peer) ปิดการเชื่อมต่อ (socket) ไปก่อนที่เราจะได้รับข้อมูลครบตามที่ต้องการ ไม่ว่าจะเป็นระหว่างรอ delimiter (read_until) หรือระหว่างอ่านข้อมูลตามจำนวนไบต์ที่กำหนด (read_exact)

Use case: ระหว่างที่ Client กำลังดาวน์โหลด Segment ขนาด 8192 bytes แล้ว Server ถูกปิดกะทันหันกลางทาง read_exact() จะ raise ConnectionClosedError พร้อมบอกว่าได้รับไปแล้วกี่ไบต์ ทำให้ Client แสดงข้อความแจ้งเตือนแทนที่จะค้าง (hang) รอข้อมูลตลอดไป

```
try:
    data = buffered_sock.read_exact(8192)
except ConnectionClosedError as e:
    print(f"[ERROR] Server disconnected: {e}")
```

class BufferedSocket — *Core wrapper class*

แก้ปัญหาพื้นฐานที่สุดของ TCP คือ TCP เป็น Byte Stream ไม่ใช่ Message Protocol การเรียก recv() หนึ่งครั้งอาจได้ข้อมูลไม่ครบหนึ่งข้อความ ได้พอดีหนึ่งข้อความ หรือได้เกินมาเป็นส่วนหนึ่งของข้อความถัดไปได้ BufferedSocket จึงเก็บ Buffer ภายในตัวเอง (self.buffer) และให้เมธอด 2 ตัวคือ read_until() และ read_exact() เพื่ออ่านข้อมูลได้อย่างถูกต้องเสมอ ไม่ว่าการมาถึงของข้อมูลจริงจะถูกแบ่งเป็นกี่ Chunk ก็ตาม

Use case: ทั้ง server.py และ client.py จะห่อ Socket ที่ได้จาก accept()/connect() ด้วย BufferedSocket ทันที แล้วใช้ตัวนี้แทนการเรียก sock.recv() ตรงๆ ตลอดทั้งโปรแกรม เพื่อให้การอ่าน Request บรรทัดเดียว, Response Header block, และ Body ที่มีความยาวตาม Content-Length ทำงานถูกต้องแม้เครือข่ายจะส่งข้อมูลมาไม่พร้อมกันเป็นก้อนเดียว

```
raw_sock = server_sock.accept()[0]
buffered = BufferedSocket(raw_sock)

# อ่านค่าขอหนึ่งบรรทัด (จนเจอ \n)
request_line = buffered.read_until(b"\n")

# อ่านข้อมูล Body ให้ครบตามจำนวนไบต์ที่ Content-Length ระบุ
body = buffered.read_exact(8192)
```

Method ภายในคลาส BufferedSocket:

read_until(delimiter: bytes) -> bytes — *Instance method*

วนเรียก recv() สะสมข้อมูลใน Buffer ไปเรื่อยๆ จนกว่าจะพบ delimiter ที่ต้องการ (b"\n" สำหรับ Request line หรือ b"\n\n" สำหรับ Response header) จากนั้น Return ข้อมูลทั้งหมด ก่อนหน้า delimiter และเก็บข้อมูลส่วนที่เกินมาไว้ใน Buffer สำหรับการอ่านครั้งถัดไป

ตัวอย่าง: buffered.read_until(b"\n\n") เพื่ออ่าน Response Header ทั้งก่อนก่อนที่จะแยกอ่าน Body ตาม Content-Length

read_exact(n: int) -> bytes — *Instance method*

คืนค่าข้อมูลให้ได้ขนาดพอดี n bytes เสมอ โดยดึงจาก Buffer ที่มีอยู่ก่อน แล้วจึงอ่านเพิ่มจาก Socket วนไปจนกว่าจะครบ n ไบต์ ใช้สำหรับรับข้อมูลที่รู้ขนาดแน่นอนล่วงหน้า เช่น MP3 Segment ที่มีขนาด SEGMENT_SIZE_BYTES

ตัวอย่าง: buffered.read_exact(headers['Content-Length']) เพื่อรับข้อมูลไฟล์เพลงให้ครบ ตามความยาวที่ Server แจ้งไว้ใน Header

7. เมธอดและฟังก์ชันของโปรโตคอล (Protocol Methods & Functions)

นอกจากคลาสหลักแล้ว protocol.py ยังมีฟังก์ชันระดับโมดูล (module-level functions) ที่แบ่งออกเป็น 4 กลุ่มตามหน้าที่: การส่งข้อมูลระดับล่าง (Low-level send/receive), การสร้าง/แปลง Request, การสร้าง/แปลง Response, และฟังก์ชันช่วยคำนวณสำหรับ ABR แต่ละฟังก์ชันอธิบายพร้อมตัวอย่างการใช้งานดังนี้:

7.1 send_all(sock, data: bytes)

ฟังก์ชันช่วยส่งข้อมูล (wrapper) ให้แน่ใจว่าข้อมูลทุกไบต์ใน data ถูกส่งออกไปจนครบ แม้ว่าระบบปฏิบัติการจะยอมรับข้อมูลเพียงบางส่วนในการเรียก send() ครั้งเดียวก็ตาม ภายในใช้ sock.sendall() ซึ่งวนส่งข้อมูลจนครบ หรือ raise ข้อผิดพลาดหากการเชื่อมต่อขาดหาย การห่อไว้ในฟังก์ชันเดียวทำให้ทุกจุดของโปรเจกต์ที่ต้องการส่งข้อมูลออก เรียกผ่านจุดเดียวกันเสมอ ง่ายต่อการดูแลและแก้ไขในอนาคต

```
request = build_request("GET_SEGMENT", 1, 3, 128)
send_all(client_socket, request)
# ส่ง b'GET_SEGMENT 1 3 128\n' ออกไปจนครบ ไม่ว่า OS จะแบ่งส่งกี่รอบก็ตาม
```

7.2 receive_exact(buffered_sock, n: int) -> bytes

ฟังก์ชันระดับโมดูลที่ห่อการเรียก buffered_sock.read_exact(n) อีกชั้นหนึ่ง เพื่อให้โค้ดฝั่งที่เรียกใช้งาน (server.py / client.py) สามารถเรียกผ่านฟังก์ชัน (function-style) แทนการเรียกเมธอดของอ็อบเจกต์โดยตรง ทำให้จุดเรียกอ่านง่ายขึ้นเมื่อไม่ต้องสนใจว่าเบื้องหลังเป็นคลาสใด และเปิดช่องให้ปรับพฤติกรรมการอ่านแบบรวมศูนย์ได้ในจุดเดียวหากต้องเพิ่ม Logic (เช่น Logging) ในอนาคต

```
content_length = int(headers["Content-Length"])
segment_bytes = receive_exact(buffered_sock, content_length)
# ได้ข้อมูล MP3 Segment ครบตามจำนวนไบต์ที่ Header ระบุไว้พอดี
```

7.3 Request Helpers (ฝั่งสร้าง/แปลงคำขอ)

กลุ่มฟังก์ชันสำหรับสร้างคำขอฝั่ง Client และแปลงคำขอที่ Server ได้รับ ให้อยู่ในรูปแบบที่โปรแกรมนำไปใช้ต่อได้ง่าย

parse_request(line: str) -> (command, params) — Function

แยกบรรทัดคำขอ "COMMAND [param1] [param2] ..." ออกเป็นชื่อคำสั่ง (ตัวพิมพ์ใหญ่เสมอ) กับลิสต์ของพารามิเตอร์ หากบรรทัดว่างจะ raise ProtocolError ทันที

```
command, params = parse_request("get_segment 1 3 128")
# command = "GET_SEGMENT", params = ["1", "3", "128"]
```

build_request(command: str, *params) -> bytes — Function

ประกอบชื่อคำสั่งและพารามิเตอร์เข้าด้วยกันเป็น 1 บรรทัด แล้วแปลงเป็น bytes พร้อมต่อท้ายด้วย \n ตามรูปแบบ Wire Format ของ SMAP

```
build_request("PLAY", 1, 128)
# -> b'PLAY 1 128\n'
```

send_request(sock, command: str, *params) -> str — Function

รวมขั้นตอน build_request() และ send_all() ไว้ในฟังก์ชันเดียว ส่งคำขอออกไปยัง Socket ทันที และคืนค่าข้อความบรรทัดที่ส่งไป (ตัดช่องว่าง/ขึ้นบรรทัดออกแล้ว) เพื่อนำไปใช้แสดงผล Log ฝั่ง Client

```
sent_line = send_request(client_socket, "BITRATE", 320)
print(f"[SEND] {sent_line}") # -> [SEND] BITRATE 320
```

receive_request(buffered_sock) -> (command, params, raw_line) — Function

อ่านคำขอหนึ่งบรรทัดจาก BufferedSocket (จนเจอ \n) แล้วเรียก parse_request() ให้ทันที คืนค่าทั้งคำสั่งที่แยกแล้ว พารามิเตอร์ และบรรทัดดิบ (raw_line) สำหรับใช้ทำ Log ฝั่ง Server

```
command, params, raw_line = receive_request(buffered_sock)
print(f"[RECEIVE] {raw_line}")
if command == "GET_SEGMENT":
    song_id, seg_index, bitrate = params
```

7.4 Response Helpers (ฝั่งสร้าง/แปลงคำตอบ)

กลุ่มฟังก์ชันสำหรับสร้าง Response Header ฝั่ง Server และแปลง Response ที่ Client ได้รับกลับมาให้เป็นข้อมูลที่ใช้งานต่อไป

build_response(status_code, headers=None, body=None) -> bytes — Function

สร้าง Response ตามรูปแบบของ SMAP: บรรทัดสถานะ (ค้นจาก STATUS_PHRASES) ตามด้วย Header แต่ละบรรทัด คั่นด้วยบรรทัดว่าง (\n\n) และต่อท้ายด้วย body หากมี (เช่น bytes ของไฟล์ MP3)

```
build_response(200, {"Content-Length": 8192})
# -> b'200 OK\nContent-Length: 8192\n\n'
build_response(404, {"Message": "Song not found"})
# -> b'404 NOT FOUND\nMessage: Song not found\n\n'
```

send_response_header(sock, status_code, headers=None) -> str — Function

สร้างเฉพาะส่วน Header block ด้วย build_response() แล้วส่งออกไปทันทีด้วย send_all() คืนค่าข้อความ Header ที่ส่งไป (เป็น str) เพื่อใช้ Log ฝั่ง Server เหมาะกับกรณีที่ Body (เช่นไฟล์เพลง) จะถูกส่งแยกเป็นอีกขั้นตอนหนึ่งภายหลัง

```
send_response_header(client_sock, 200, {"Content-Length":
len(segment_data)})
client_sock.sendall(segment_data) # ส่ง Body (Raw MP3 bytes) ตามมาแยกต่างหาก
```

receive_response_header(buffered_sock) -> (status_code, status_phrase, headers, raw_text) — Function

อ่าน Response Header block ทั้งก้อนจนเจอ \n\n แล้วแยกบรรทัดสถานะออกเป็นรหัส (int) กับข้อความสถานะ และแยก Header แต่ละบรรทัดเป็น dict (key: value) คืนค่าทั้งหมดพร้อมข้อความดิบ (raw_text) สำหรับ Debug/Log

```
status_code, status_phrase, headers, raw =
receive_response_header(buffered_sock)
```

```

if status_code == 200:
    body = receive_exact(buffered_sock, int(headers["Content-Length"]))
else:
    print(f"[ERROR] {status_code} {status_phrase}:
{headers.get('Message')}")

```

7.5 to_json_bytes(obj) -> bytes

แปลงอ็อบเจกต์ Python (เช่น dict ของแคตตาล็อกเพลง) ให้เป็น bytes รูปแบบ JSON (เข้ารหัส UTF-8, จัดรูปแบบด้วย indent=2 ให้อ่านง่ายเวลา Debug) ใช้เป็น Body ของ Response สำหรับ คำสั่ง LIST ที่ต้องส่งรายชื่อเพลงและ Bitrate ที่รองรับกลับไปให้ Client

```

catalog = {"1": {"title": "Song A", "bitrates": [64, 128, 192, 320]}}
body = to_json_bytes(catalog)
send_all(sock, build_response(200, {"Content-Length": len(body)}, body))

```

7.6 calculate_throughput_kbps(num_bytes, elapsed_seconds) -> float

คำนวณความเร็วในการดาวน์โหลดจากจำนวนไบต์ที่ได้รับจริงและเวลาที่ใช้จริง (วัดด้วย time.perf_counter() ฝั่ง Client) ด้วยสูตร $(\text{bytes} * 8) / (\text{elapsed_seconds} * 1000)$ ผลลัพธ์เป็น หน่วย kbps มีการป้องกันการหารด้วยศูนย์โดยกำหนด elapsed_seconds ขั้นต่ำไว้ที่ 0.001 วินาที

```

start = time.perf_counter()
segment = receive_exact(buffered_sock, SEGMENT_SIZE_BYTES)
elapsed = time.perf_counter() - start
throughput_kbps = calculate_throughput_kbps(len(segment), elapsed)
# เช่น ได้ 8192 bytes ใน 0.05 วินาที -> throughput_kbps ~= 1310.7 kbps

```

7.7 select_bitrate(preferred_bitrate, available_throughput_kbps) -> (bitrate, reason)

อัลกอริทึมหลักของระบบ ABR ใช้ตัดสินใจเลือก Bitrate สำหรับ Segment ถัดไป โดยมีกฎ 3 ข้อ เรียงตามลำดับความสำคัญ: (1) ห้ามเลือก Bitrate สูงกว่าค่า preferred_bitrate ที่ผู้ใช้กำหนดไว้ เด็ดขาด ถือเป็นเพดานตายตัว (2) ในบรรดา Bitrate ที่ preferred_bitrate อนุญาตให้เลือกค่า สูงสุดที่ยังไม่เกิน available_throughput_kbps ซึ่งควรเป็นค่าที่คูณ SAFETY_FACTOR มาแล้วจาก ฝั่งผู้เรียก (3) หากแม้แต่ Bitrate ต่ำสุด (64 kbps) ก็ยังไม่สามารถรองรับได้ ก็ยังคงคืนค่า 64 kbps เป็นค่าต่ำสุดเสมอ ไม่ปฏิเสธการสตรีม ฟังก์ชันคืนค่าเป็นคู่ (bitrate, reason) โดย reason มี 3 ค่าคือ OPTIMAL (ได้ Bitrate สูงสุดของระบบ), USER_PREFERENCE_LIMIT (ถูกจำกัดโดย ค่าที่ผู้ใช้ตั้งไว้ ไม่ใช่โดยเครือข่าย), และ NETWORK_LIMITED (เครือข่ายไม่ไหวจึงต้องลดระดับลง)

```

avg_kbps = sum(recent_throughputs[-THROUGHPUT_HISTORY_SIZE:]) /
THROUGHPUT_HISTORY_SIZE
safe_kbps = avg_kbps * SAFETY_FACTOR
next_bitrate, reason = select_bitrate(preferred_bitrate=320,
available_throughput_kbps=safe_kbps)
print(f"[ADAPTATION] Next bitrate: {next_bitrate} kbps ({reason})")
# เช่น safe_kbps = 150 -> next_bitrate = 128, reason = 'NETWORK_LIMITED'

```

8. จุดเด่นและข้อเปรียบเทียบของ Protocol

เพื่อให้เห็นถึงศักยภาพและการออกแบบเชิงเครือข่าย (Network Engineering) โพรโทคอล SMAP มีประสิทธิภาพและการทำงานที่เหนือกว่าเมื่อเทียบกับโปรโตคอลทั่วไปอย่าง HTTP ในบริบทนี้ดังนี้:

8.1 Lightweight & Purpose-Built (ใช้ Overhead ต่ำกว่า HTTP)

HTTP เป็นโปรโตคอลแบบ General-purpose ที่มาพร้อมกับ Header จำนวนมาก (เช่น User-Agent, Accept, Cookies) ซึ่งสิ้นเปลือง Bandwidth อย่างมาก แต่ SMAP ถูกออกแบบมาให้มีเฉพาะ Header ที่จำเป็นสำหรับการสตรีมสื่อ (เช่น Song-ID, Segment-ID, Bitrate) ทำให้ประหยัด Overhead และเพิ่ม Performance ได้อย่างชัดเจนเมื่อมีการดาวน์โหลด Segment ขนาดเล็กซ้ำๆ

8.2 Stateful Connection ลดการสื่อสาร (Low Latency)

HTTP ตามมาตรฐานเป็น Stateless (ต้องส่ง Token ยืนยันสถานะทุกครั้ง) แต่ SMAP ถูกออกแบบให้เป็น Stateful ฝั่ง Server จะจดจำ Preferred Bitrate และ Network Profile ของผู้ใช้ไว้ ทำให้คำสั่ง GET_SEGMENT มีขนาดเล็กกะทัดรัด ไม่ต้องส่ง Parameter ส่วนตัวซ้ำ ช่วยลด RTT (Round-Trip Time) ในการสื่อสาร

8.3 Single Persistent Connection ลดเวลา TCP Handshake

แทนที่จะต้องเปิดและปิด Connection ใหม่ตลอดเวลา (เหมือน HTTP/1.0 รุ่นเก่า) SMAP เปิด TCP เพียง 1 เส้นแล้วใช้งานแบบ Persistent ตลอดเซสชัน วิธีนี้หลีกเลี่ยงกระบวนการ 3-Way Handshake และ TCP Slow Start ช้าซ้อน ทำให้การสตรีมมีความต่อเนื่องสูง

8.4 Built-in Adaptive Bitrate (ABR) และวิธีการวัดผลที่เป็นรูปธรรม

โปรโตคอลมีการกำหนดมาตรฐานการส่งเป็น Segment ขนาดคงที่ (8192 Bytes)

- วิธีการวัดประสิทธิภาพ (Measurement): โปรแกรมวัดระยะเวลาในการดาวน์โหลดจริงและคำนวณ Throughput ออกมาเป็น kbps โดยตรงผ่านสูตร $(\text{bytes} * 8) / (\text{elapsed_seconds} * 1000)$
- อัลกอริทึมการตัดสินใจ: SMAP จะนำค่าเฉลี่ย Throughput จาก 3 Segment ล่าสุดมาคูณกับค่า Safety Factor (0.70) เพื่อความผันผวนของเครือข่าย ก่อนนำไปตัดสินใจเลือก Bitrate ของ Segment ถัดไป ทำให้โปรโตคอลนี้สามารถหลีกเลี่ยงอาการเสียงกระตุก (Buffer Underrun) ได้อย่างชาญฉลาด

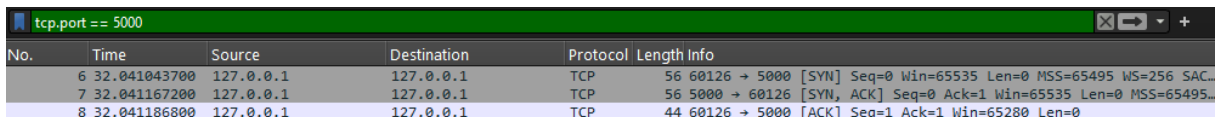
9. ข้อจำกัดของ Protocol และระบบ (Limitations)

- จุดประสงค์เพื่อการศึกษา (Educational Purpose): ตัวโปรโตคอลนี้เป็นเพียง Custom Protocol ไม่ได้มาตรฐานตามโปรโตคอล Streaming ของอุตสาหกรรม (เช่น MPEG-DASH หรือ HLS) และไม่มีไฟล์ Manifest ที่แท้จริง
- การแบ่ง Segment ไม่ได้อิงตามเวลา (Not Frame-Accurate): การตัดแบ่งไฟล์ (Segmentation) อาศัยการตัดตามจำนวนไบต์แบบขนาดคงที่ (Fixed-size byte-range) ไม่ได้ตัดตามเฟรมของเสียงเหมือนโปรโตคอลมาตรฐาน ทำให้เวลาปรับเปลี่ยน Bitrate กลางคันอาจทำให้ตำแหน่งของเสียง (Audio Timestamp) คลาดเคลื่อนเล็กน้อย
- ไม่สามารถแทรกแซงการส่งข้อมูลที่กำลังดำเนินการอยู่ (Synchronous Transfer): คำสั่ง STOP จะไปรีเซ็ตสถานะบน Server ได้ก็ต่อเมื่อรอให้การโหลด Segment ปัจจุบันเสร็จสิ้นเท่านั้น ไม่สามารถขัดจังหวะ (Interrupt) การส่งข้อมูลที่กำลังสตรีมอยู่กลางคันได้
- ข้อจำกัดด้านการขยายตัว (Scalability): สถาปัตยกรรมของแอปพลิเคชันที่ใช้แบบ Thread-per-client นั้นง่ายต่อการทำความเข้าใจ แต่อาจเกิดปัญหาเรื่อง Scalability และไม่สามารถรองรับการใช้งานพร้อมกันจำนวนมาก (Large Scale) ได้เมื่อเทียบกับสถาปัตยกรรม Event-loop หรือ Process-pool
- ข้อจำกัดด้านพอร์ต (Single Server Instance): Server สามารถรันได้เพียง 1 Instance ต่อหนึ่ง Host/Port เท่านั้น หากพยายามเปิดซอฟต์แวร์ซ้ำโดยใช้พอร์ตเดิมจะเกิดปัญหาเชื่อมต่อไม่ได้
- การเพิ่มเพลงใหม่ ผู้ดูแลระบบต้องเพิ่มไฟล์เพลงลงใน Directory ของ Server โดยตรง และต้องเตรียมไฟล์เพลงในแต่ละ Bitrate ที่ระบบรองรับไว้ล่วงหน้า
- การเพิ่มเพลงหรือ Bitrate ใหม่ขณะ Server กำลังทำงาน จำเป็นต้องทำการ Restart Server เนื่องจากระบบจะทำการ Scan Media Catalog และสร้าง Media Segment สำหรับเพลงและ Bitrate ต่าง ๆ เพียงครั้งเดียวในขั้นตอนการเริ่มต้น Server ดังนั้น เพลงหรือ Bitrate ที่เพิ่มเข้ามาใหม่จะยังไม่ถูกตรวจพบจนกว่า Server จะถูก Restart

10. การตรวจสอบการทำงานด้วย Wireshark

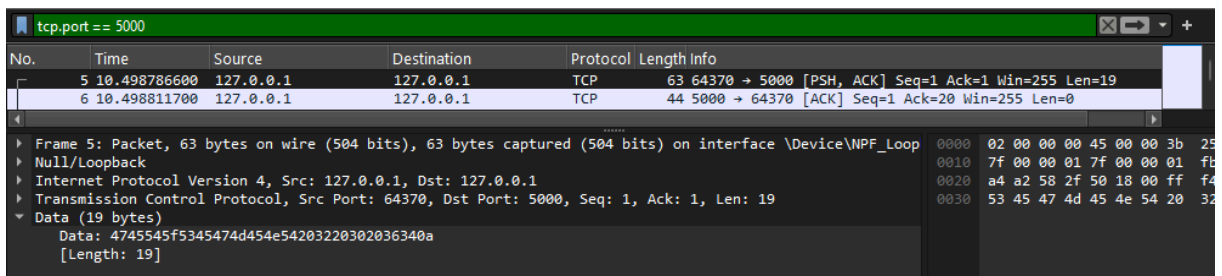
สามารถจับแพ็กเก็ต (Packet Sniffing) เพื่อตรวจสอบและยืนยันการทำงานของโปรโตคอลได้ดังนี้:

- ดักจับ Interface ที่ใช้งาน (เช่น Loopback หากรันบนเครื่องตนเอง) และตั้งค่า Filter ให้แสดงเฉพาะ tcp.port == 5000
- สังเกตกระบวนการ 3-Way Handshake (SYN, SYN-ACK, ACK) จะเห็นว่าเกิดขึ้นเพียงครั้งเดียวต่อหนึ่ง Client



No.	Time	Source	Destination	Protocol	Length	Info
6	32.041043700	127.0.0.1	127.0.0.1	TCP	56	60126 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK...
7	32.041167200	127.0.0.1	127.0.0.1	TCP	56	5000 → 60126 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495...
8	32.041186800	127.0.0.1	127.0.0.1	TCP	44	60126 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0

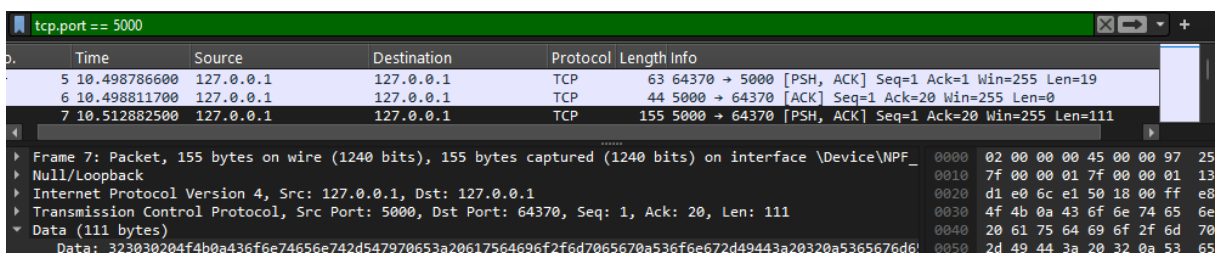
- เมื่อตรวจสอบ Payload จะเห็นข้อมูลบรรทัดคำสั่งที่เป็น Text Request ชัดเจน (เช่น GET_SEGMENT 1 0 128)



No.	Time	Source	Destination	Protocol	Length	Info
5	10.498786600	127.0.0.1	127.0.0.1	TCP	63	64370 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
6	10.498811700	127.0.0.1	127.0.0.1	TCP	44	5000 → 64370 [ACK] Seq=1 Ack=20 Win=255 Len=0

Frame 5: Packet, 63 bytes on wire (504 bits), 63 bytes captured (504 bits) on interface \Device\NPF_...
 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 Transmission Control Protocol, Src Port: 64370, Dst Port: 5000, Seq: 1, Ack: 1, Len: 19
 Data (19 bytes)
 Data: 4745545f5345474d454e542032203036340a
 [Length: 19]

- เมื่อดูการตอบกลับ (Response) จะเห็น Text Header แจ้งสถานะและความยาวไฟล์ (Content-Length) และตามด้วยชุดข้อมูลไบนารีแบบ Binary (Raw MP3 data) ทันทีหลังบรรทัดว่าง



No.	Time	Source	Destination	Protocol	Length	Info
5	10.498786600	127.0.0.1	127.0.0.1	TCP	63	64370 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
6	10.498811700	127.0.0.1	127.0.0.1	TCP	44	5000 → 64370 [ACK] Seq=1 Ack=20 Win=255 Len=0
7	10.512882500	127.0.0.1	127.0.0.1	TCP	155	5000 → 64370 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=111

Frame 7: Packet, 155 bytes on wire (1240 bits), 155 bytes captured (1240 bits) on interface \Device\NPF_...
 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 Transmission Control Protocol, Src Port: 5000, Dst Port: 64370, Seq: 1, Ack: 20, Len: 111
 Data (111 bytes)
 Data: 323030204f4b0a436f6e74656e742d547970653a20617564696f2f6d7065670a536f6e672d49443a20320a5365676d6f
 [Length: 111]

- สามารถดูกราฟ Throughput บน Wireshark (เมนู I/O Graphs) เทียบกับค่า Network Measurement ที่พิมพ์ทางฝั่งไคลเอนต์เพื่อความถูกต้องของการวัดผลเครือข่าย

